Chapitre 4: Gestion des Processus et Ordonnancement



Objectifs du chapitre

1. Illustrer le modèle à cinq états d'un processus et expliquer chaque transition
2. Expliquer le rôle du **PCB** (bloc de contrôle) et du changement de contexte
3. Implémenter fork() / exec() / wait() en C et décrire la hiérarchie des processus
4. Piloter les processus avec ps, top, pstree, kill
5. Calculer les métriques d'ordonnancement pour **FCFS**, **SJF**, **RR**, **Priorité**
6. Changer les prioritées nice et renice

Section 1 : Le concept de processus

Programme vs processus → organisation mémoire → états → PCB → aperçu des threads

Programme vs processus

- **Programme** : entité passive — un fichier contenant une liste d'instructions, stocké sur le disque (l'exécutable)
- **Processus** : entité active — un programme en cours d'exécution, avec un compteur ordinal et un jeu de registres
- Le processus est **l'unité de travail** du système informatique moderne

Un programme devient un processus quand l'exécutable est chargé en mémoire

- Plusieurs processus peuvent exécuter le même programme, chacun avec ses propres données

Le processus : entité active

- Le processus se décompose en trois éléments : **code**, **activité courante**, **ressources**
- L'activité courante est portée par le compteur ordinal et les registres du processeur
- Les ressources incluent la mémoire, les fichiers ouverts et les périphériques
- Deux copies d'un même programme sont **deux séquences d'exécution distinctes**
- **Exemple** : deux fenêtres du navigateur → deux PID, deux PCB, mais un code partagé
- Un processus peut lui-même servir d'environnement d'exécution (ex. la JVM qui exécute du bytecode)

Organisation mémoire d'un processus

- **Texte** : le code exécutable, en lecture seule
- **Données** : les variables globales (initialisées et non initialisées)
- **Tas (heap)** : la mémoire allouée dynamiquement (malloc), croît vers les adresses hautes
- **Pile (stack)** : paramètres, variables locales, adresses de retour ; croît vers les adresses basses

- Texte et données ont une taille fixe ; le tas et la pile sont dynamiques et ne doivent jamais se chevaucher

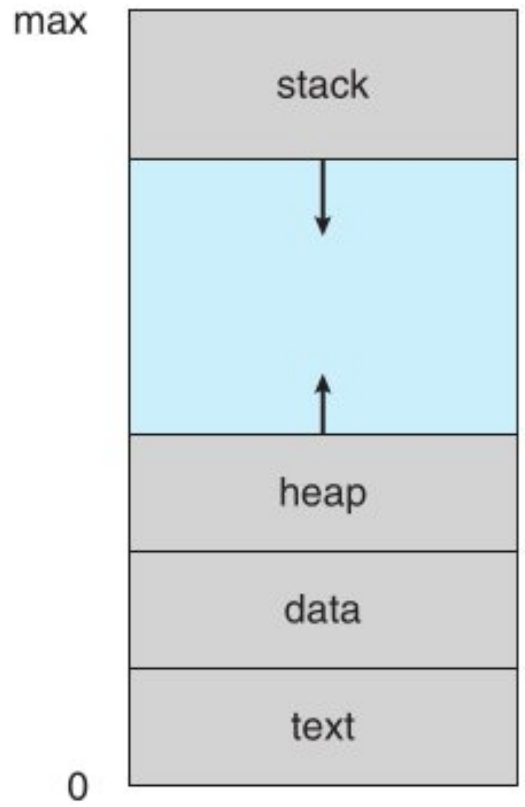


Fig 1. *Layout of a process in memory.*

Exemple: Organisation de la mémoire d'un programme C

- **Arguments (**argc**,** argv**)**: Stockés au sommet de la mémoire (adresses hautes).
- **Pile (Stack)** : Variables locales (int *values, int i). Grandit vers le bas.
- **Tas (Heap)** : Allocation dynamique (malloc). Grandit vers le haut.
- **Données non initialisées (BSS / Uninitialized Data)** : Variables globales non initialisées (int x;).
- **Données initialisées (Initialized Data)** : Variables globales initialisées (int y = 15;).
- **Code / Texte (Text Segment)** : Instructions exécutables du programme (adresses basses).

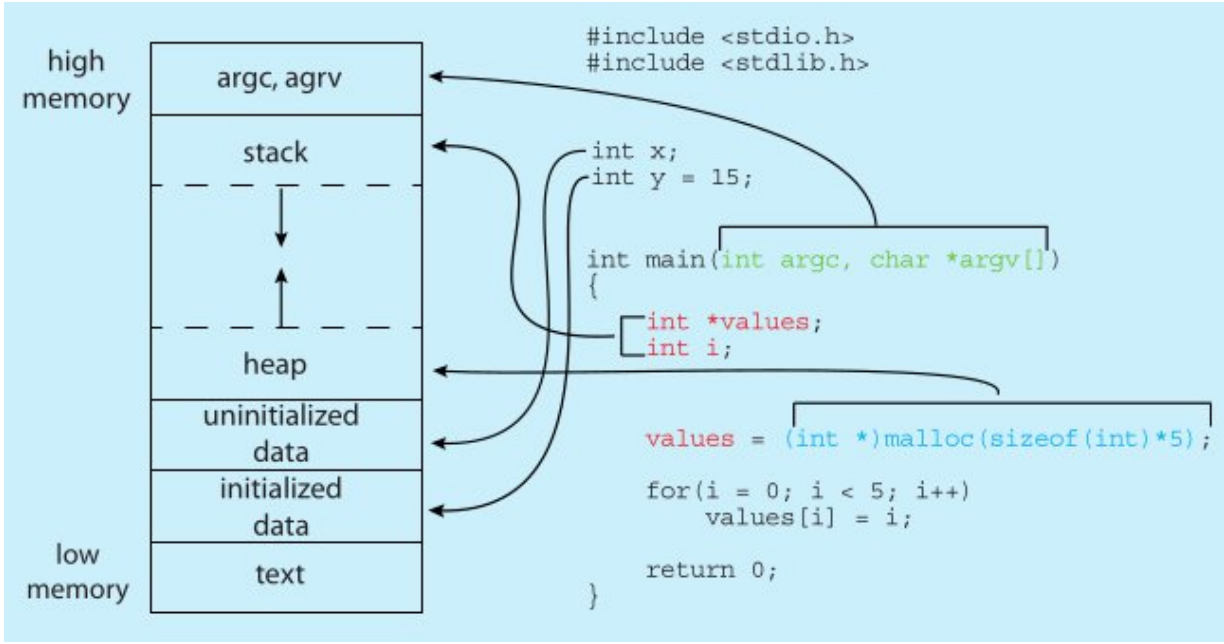


Fig 2. *Memory layout of a C program*

La commande size affiche les tailles : **text, data, bss**

Les états d'un processus

- **Nouveau (new)** : le processus est en cours de création
- **Prêt (ready)** : le processus est prêt à s'exécuter et attend le processeur

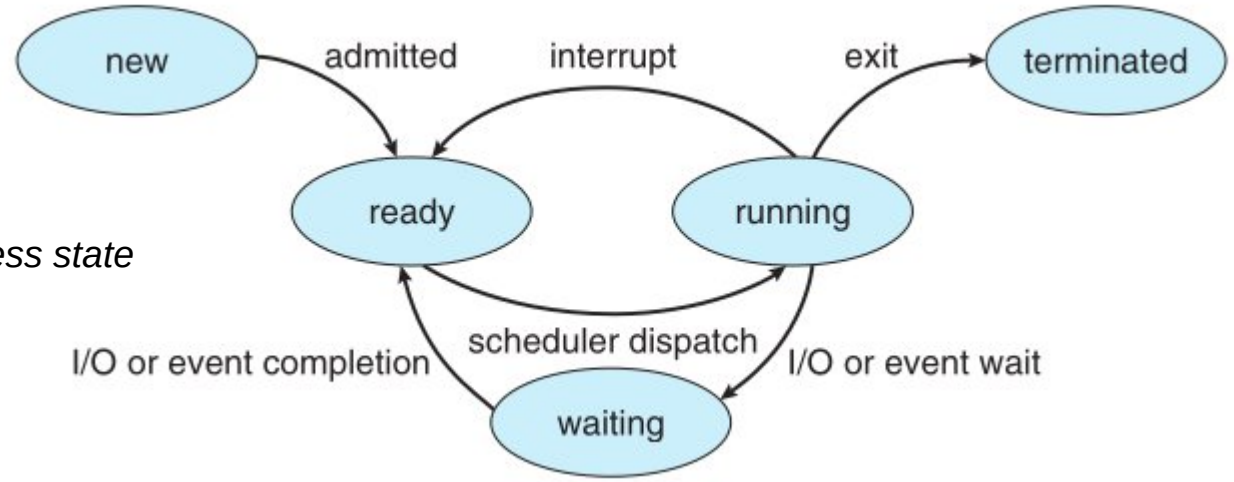


Fig 3. *Diagram of process state*

- **En exécution (running)** : les instructions du processus s'exécutent
- **En attente (waiting)** : le processus attend un événement (fin d'E/S, signal)
- **Terminé (terminated)** : l'exécution est achevée

Le diagramme de transitions d'états

1. nouveau → prêt : admission
2. prêt → en exécution : élection (dispatch) par l'ordonnanceur

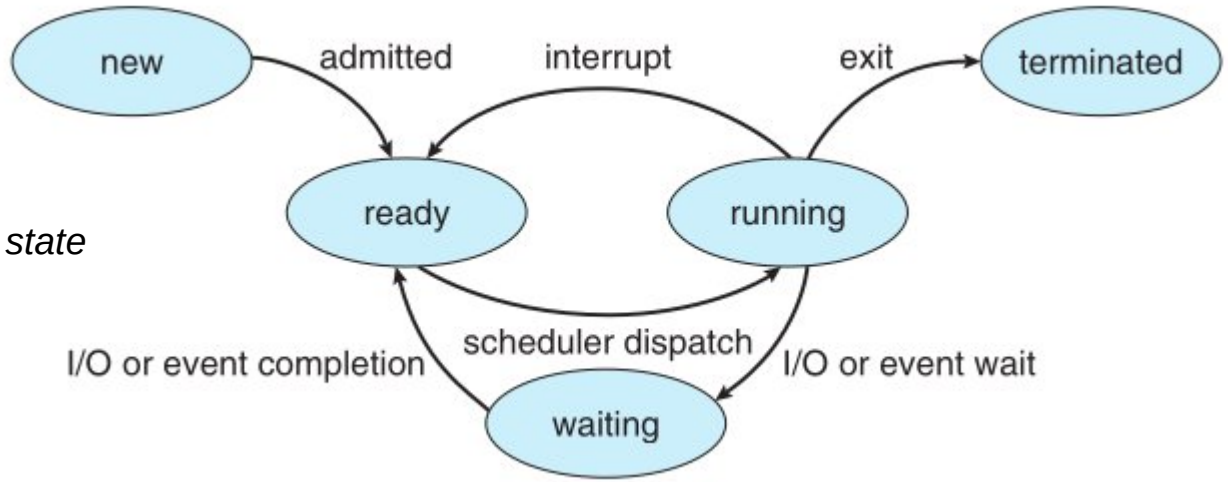


Fig 3. *Diagram of process state*

3. en exécution → prêt : interruption ou fin de quantum
4. en exécution → en attente : demande d'E/S ou attente d'un événement
5. en attente → prêt : fin d'E/S (jamais directement vers l'exécution)

Le PCB (bloc de contrôle de processus)

- **PCB (Process Control Block)** : structure de données qui représente un processus dans le noyau
- Aussi appelé « **bloc de contrôle de tâche** » ; sous Linux, c'est la `task_struct`
- Il regroupe **tout** ce qu'il faut pour **démarrer**, **interrompre**, puis **reprendre** un processus
- Un **PCB** par processus, résidant en permanence dans l'espace mémoire du noyau



Fig 4. *Process control block (PCB)*

Contenu du PCB

1. **État du processus**: L'état peut être : nouveau, prêt, en cours d'exécution, en attente, arrêté, etc.
2. **Compteur de programme**: Le compteur indique l'adresse de la prochaine instruction à exécuter pour ce processus.
3. **Registres du processeur**: Ils comprennent les accumulateurs, les registres d'index, les pointeurs de pile et les registres d'usage général… Ces informations d'état doivent être sauvegardées lors d'une interruption afin de permettre la reprise correcte du processus.
4. **Informations d'ordonnancement du processeur:** Ces informations incluent la priorité du processus, des pointeurs vers les files d'attente d'ordonnancement et tout autre paramètre d'ordonnancement.



Fig 4. *Process control block (PCB)*

Contenu du PCB

5. **Informations de gestion de la mémoire:** Ces informations peuvent inclure, par exemple, les valeurs des registres de base et de limite ainsi que les tables de pages ou les tables de segments, selon le système de mémoire utilisé par le SE.
6. **Informations de comptabilité:** Ces informations comprennent la quantité de temps processeur et de temps réel consommée, les limites de temps, les numéros de compte, les numéros de tâche ou de processus, etc.
7. **Informations d'état des entrées/sorties:** Ces informations incluent la liste des périphériques d'entrée/sortie alloués au processus, la liste des fichiers ouverts, etc.



Fig 4. *Process control block (PCB)*

Aperçu : les threads

- Jusqu'ici, un processus = un seul fil d'exécution (une instruction à la fois)
- Un thread (fil) est un fil d'exécution au sein d'un processus
- Un processus multithread peut faire plusieurs tâches simultanément (ex. saisie + correction orthographique)
- Sur un multicœur, plusieurs threads s'exécutent réellement en parallèle
- Le PCB s'étend alors pour stocker l'information de chaque thread

Section 2 : Files d'attente et changement de contexte

Files des prêts et files d'attente → diagramme de files → changement de contexte → coût

Multiprogrammation vs Temps Partagé

Multiprogrammation:

- **Objectif** : Maximiser l'utilisation du processeur (garder le CPU toujours occupé).
- **Degré de multiprogrammation** : Nombre total de processus actuellement chargés en mémoire.

Temps Partagé (Time Sharing)

- **Objectif** : Commuter le CPU très fréquemment entre les processus.
- **Bénéfice** : Permet une interactivité en temps réel avec l'utilisateur.

Ordonnancement et Gestion des Cœurs CPU

Le Rôle de l'Ordonnanceur (Process Scheduler)

- Sélectionne un processus prêt en mémoire et l'alloue à un cœur CPU.

Règle d'Exécution:

- 1 cœur CPU = 1 seul processus à la fois.
- **Système monocœur** : Un seul processus s'exécute réellement à un instant t.
- **Système multicœur** : Exécution parallèle de plusieurs processus.
- Si Nombre de processus > Nombre de cœurs → Les processus en excès attendent la libération d'un cœur.

Les files d'attente

- **File des prêts (ready queue)** : les processus prêts à s'exécuter, en attente du CPU
- **Files d'attente (wait queues)** : les processus qui attendent un événement précis (fin d'E/S)
- Implémentées typiquement comme des **listes chaînées de PCB**
- Un en-tête de file pointe vers le premier PCB ; chaque PCB pointe vers le suivant

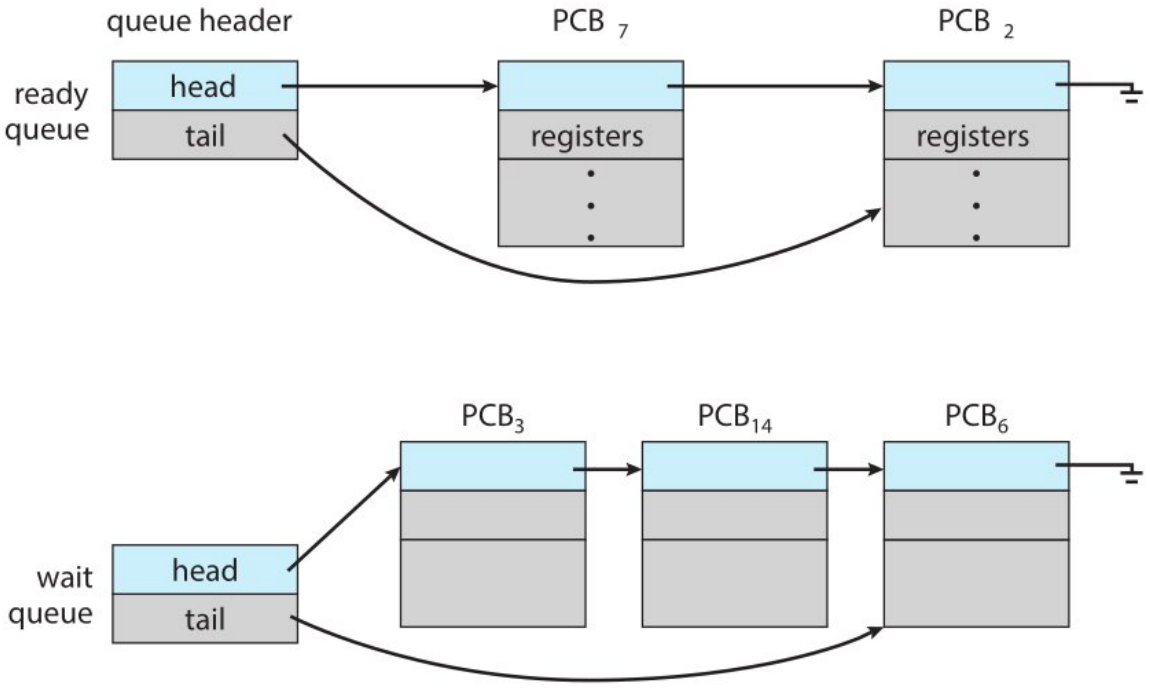


Fig 5. *The ready queue and wait queues*

Le diagramme de files (queueing diagram)

- Nouveau processus → file des prêts → CPU
- Une fois sur le CPU, le processus peut : demander une E/S, créer un enfant, ou être interrompu
- Fin de quantum → retour en file des prêts
- Fin d'E/S ou fin d'enfant → retour en file des prêts
- Le cycle se poursuit jusqu'à la terminaison (libération du PCB et des ressources)

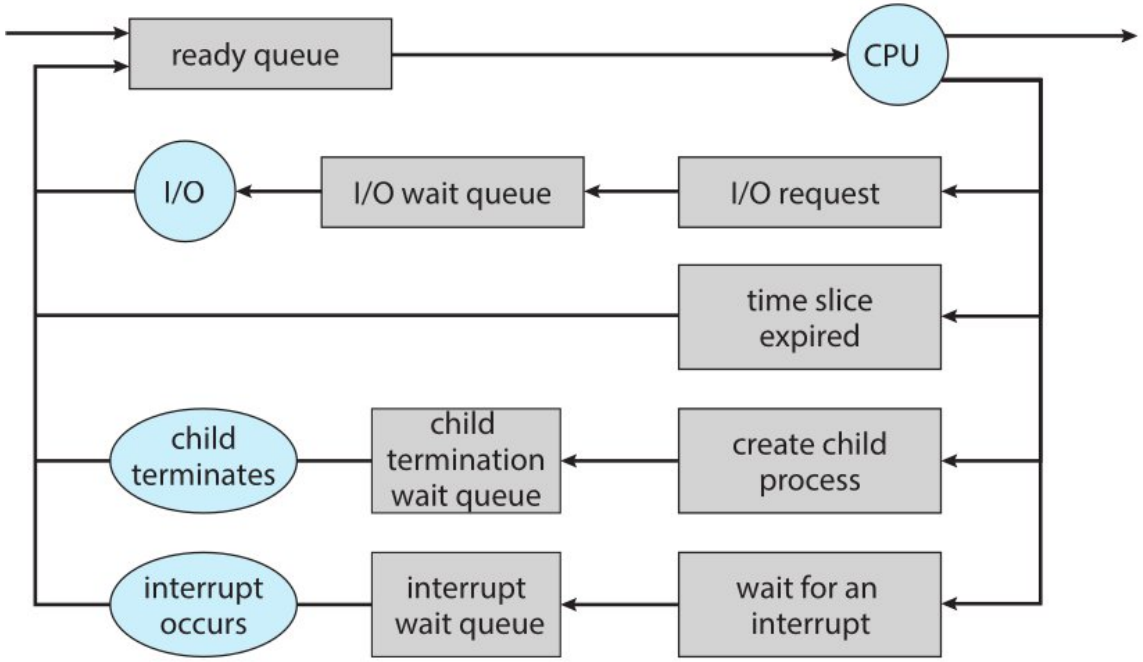


Fig 6. *Queueing-diagram representation of process scheduling*

Le changement de contexte

- **Changement de contexte**: Sauvegarder l'état du processus courant, puis restaurer celui d'un autre
- Le contexte est représenté dans le PCB : registres, état, compteur ordinal, information mémoire
- Déclenché par une **interruption** ou un **appel système**
- Le noyau sauvegarde le contexte de l'ancien processus dans son PCB, puis charge celui du nouveau
- Le temps de bascule est du **surcoût** pur : le système ne fait aucun travail utile pendant ce temps

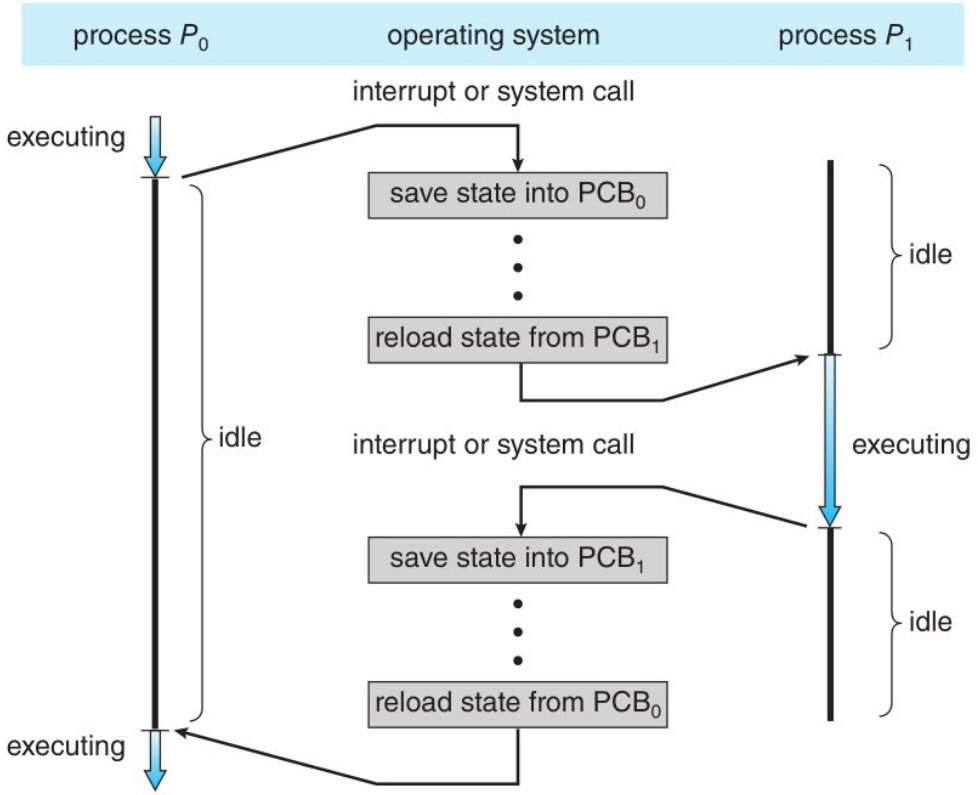


Fig 7. *Context switch from process to process*

Coût du changement de contexte

- Vitesse typique : **quelques microsecondes**.
- Dépend de : vitesse de la mémoire, nombre de registres à copier, instructions spéciales (chargement global)
- Certains processeurs offrent **plusieurs jeux de registres** : la bascule se réduit à changer un pointeur
- Coût caché : la gestion mémoire avancée invalide le **cache** et la **TLB** à chaque bascule
- Observable sous Linux : `/proc/PID/status` (bascule volontaire/non volontaire) et `vmstat` (Report virtual memory statistics)

Section 3 : Création et terminaison des processus

fork() → exec() → wait()/exit() → zombies et orphelins → arbre des processus

Création de processus et identifiant (pid)

Processus parents et enfants

- Un processus peut créer plusieurs nouveaux processus pendant son exécution
- Processus créateur = parent ; nouveaux processus = enfants
- Les enfants peuvent à leur tour créer des processus → formation d'un arbre de processus

Identifiant de processus (pid)

- Utilisé par UNIX, Linux et Windows pour identifier les processus
- Nombre entier unique pour chaque processus du système
- Sert d'index pour accéder aux attributs d'un processus dans le noyau
- **systemd** (pid = 1) : processus parent racine de tous les processus utilisateur, premier processus utilisateur créé au démarrage

fork() : créer un processus

- fork() : **appel système** qui crée un processus enfant en **dupliquant** le parent
- L'enfant est une copie de **l'espace d'adressage** du parent (même code, mêmes données)
- Les deux processus reprennent l'exécution à l'instruction qui suit le fork()
- Valeur de retour : **0 chez l'enfant**, le **PID de l'enfant chez le parent**
- Une valeur **négative** signale une **erreur** (ex. manque de ressources)

fork() en action

- pid = fork(); puis tester la valeur de retour:
  - pid < 0 → erreur.
  - pid == 0 → code de l'enfant.
  - pid > 0 → code du parent (pid de l'enfant)
- l'enfant appelle exec,
- le parent appelle wait.
- Chaque processus possède sa propre copie des données (modifier chez l'un n'affecte pas l'autre).
- L'enfant hérite de certains attributs : fichiers ouverts, privilèges, attributs d'ordonnancement

```
#include <sys/types.h>
#include <stdio.h>
#include <unistd.h>

int main()
{
pid_t pid;

   /* fork a child process */
   pid = fork();

   if (pid < 0) { /* error occurred */
      fprintf(stderr, "Fork Failed");
      return 1;
   }
   else if (pid == 0) { /* child process */
      execlp("/bin/ls","ls",NULL);
   }
   else { /* parent process */
      /* parent will wait for the child to complete */
      wait(NULL);
      printf("Child Complete");
   }

   return 0;
}
```

Fig 9. *Creating a separate process using the UNIX fork() system call.*

exec() : remplacer l'image

- exec() : charge un nouveau programme dans le processus courant
- Il **détruit l'image mémoire** actuelle et la remplace par le binaire demandé
- Il ne crée **pas** de nouveau processus : le PID reste le même
- Il ne **retourne pas** en cas de succès — il n'y a plus d'ancien programme où revenir
- **Variantes** : execlp, execvp, execve (différences sur le passage des arguments et de l'environnement) (Voir: man exec)

wait() et exit() : terminaison

- exit() : termine le processus, libère ses ressources et renvoie un **statut** au parent
- wait() : suspend le parent jusqu'à la fin d'un enfant, et récupère son statut
- pid = wait(&status) renvoie **le PID de l'enfant terminé**
- Le parent peut aussi continuer à s'exécuter en **parallèle** de ses enfants, sans attendre
- En C, exit() est appelé implicitement par la bibliothèque d'exécution en fin de main

Terminaison de processus : autres circonstances

- Un processus peut terminer un autre processus via un appel système
- En général, seul le parent du processus à terminer peut invoquer cet appel. Sinon, un utilisateur ou une application fautive pourrait tuer arbitrairement les processus d'un autre utilisateur.
- Le parent doit connaître l'identité de ses enfants. Lorsqu'un processus crée un enfant, l'identité du nouvel enfant est transmise au parent.
- Un parent peut terminer un enfant si :
  - l'enfant a dépassé l'utilisation de certaines ressources allouées ;
  - la tâche assignée à l'enfant n'est plus nécessaire ;
  - le parent se termine et l'OS n'autorise pas l'enfant à continuer.

Processus Zombies et orphelins

- **Zombie** : processus terminé dont le parent n'a pas encore appelé `wait()`
- Ses ressources sont libérées, mais son entrée dans la table des processus subsiste (pour garder le statut)
- On **ne peut pas tuer** un zombie : il est déjà mort (état `Z` dans `ps`)
- **Orphelin** : enfant dont le parent est mort avant lui
- L'orphelin est adopté par `systemd` (PID 1), qui appelle périodiquement `wait()`

Exemple d'arbre de processus Linux

- Chaque processus a un **parent** (PPID) et peut créer des **enfants**
- systemd (PID 1) est la **racine** de tous les processus utilisateur
- De systemd descendent les services : logind, sshd, etc.
- pstree -p affiche cette hiérarchie directement
- Le parent peut restreindre l'enfant à un **sous-ensemble de ses ressources**

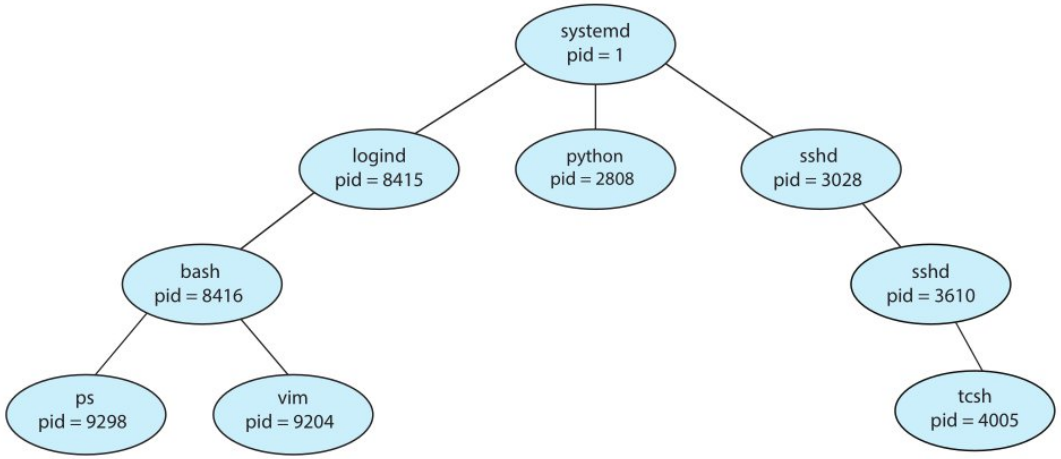


Fig 8. *A tree of processes on a typical Linux system.*

Section 4 : Observer et piloter les processus sous Linux

ps → top → pstree/pgrep → signaux et kill → premier/arrière-plan

La commande ps

- ps (process status) affiche l'état des processus (**Voir**: man ps)
- Trois familles d'options : **Unix98** (-), **BSD** (sans -), **GNU** (--)
- ps aux (BSD) ou ps -ef (Unix) : afficher tous les processus
- ps -eLf : afficher aussi les threads · ps -u user : les processus d'un utilisateur
- ps lit l'information dans le noyau via le pseudo-système de fichiers /proc

Interpréter la sortie de ps

- **PID** : identifiant du processus · **PPID** : identifiant de son parent
- **%CPU** / **%MEM** : usage processeur et mémoire à l'instant de la mesure
- **VSZ** / **RSS** : mémoire virtuelle totale / mémoire résidente en RAM
- **STAT** : état (`R` running, `S` sleeping, `Z` zombie, `T` stopped)
- **TIME** : temps CPU total consommé — utile pour repérer un processus anormal

top et htop

- **top** : vue dynamique, mise à jour toutes les **~5 s**, tri par défaut sur %CPU
- La **charge moyenne (load average)** en première ligne : demande en temps CPU
- Touches interactives : k tuer, r renicer, M trier par mémoire, q quitter
- top -p pid : surveiller des **PID** précis
- **htop** : variante plus riche et colorée (**Installer avec**: sudo apt install htop)
- Une charge de 1.0 ≈ un cœur saturé ; une charge de 0.0 = aucun processus en attente

top: Première ligne : Informations générales

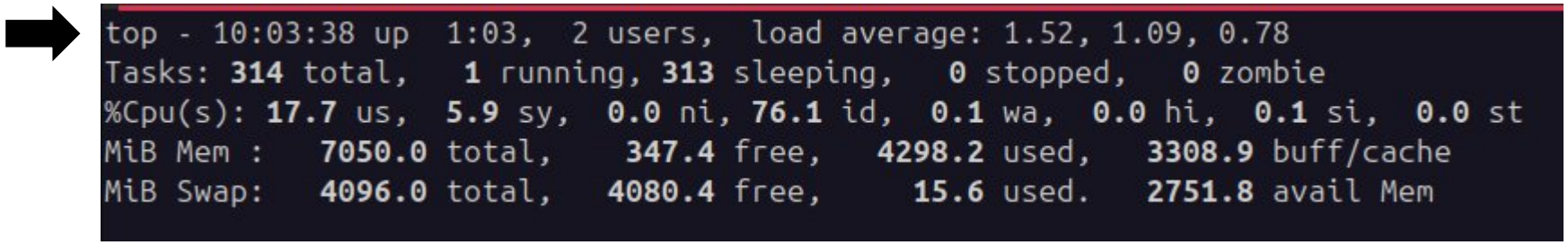


Fig 9. *top command headers*

- 10:03:38 : L'heure actuelle du système.
- up 1:03 : Le système est allumé depuis 1 heure et 3 minutes.
- 2 users : 2 utilisateurs sont actuellement connectés.
- load average: 1.52, 1.09, 0.78 : La charge moyenne du système sur les 1, 5 et 15 dernières minutes.

top: Deuxième ligne : Tâches (Processus)

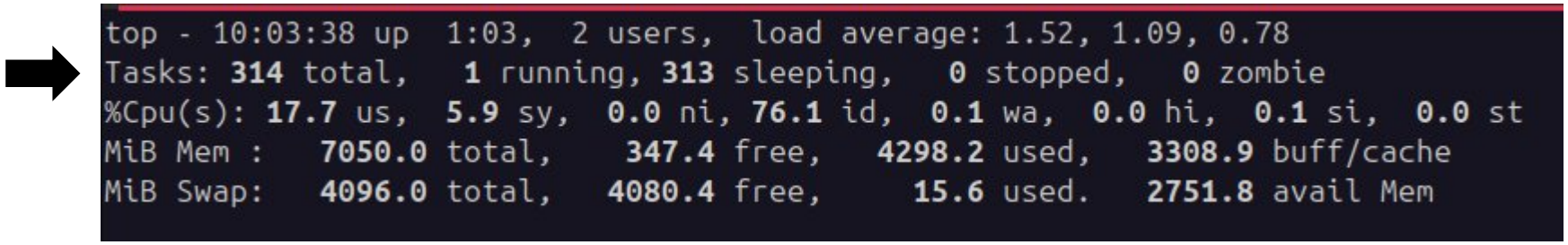


Fig 9. *top command headers*

- Tasks : Tâches (ou processus).
- 314 total : Nombre total de processus suivis.
- 1 running : 1 processus est en cours d'exécution.
- 313 sleeping : 313 processus sont en attente (endormis).
- 0 stopped : 0 processus arrêté (suspendu manuellement).
- 0 zombie : 0 processus zombie (processus terminé dont le parent n'a pas récupéré le statut de sortie).

top: Troisième ligne : Utilisation du processeur (CPU)

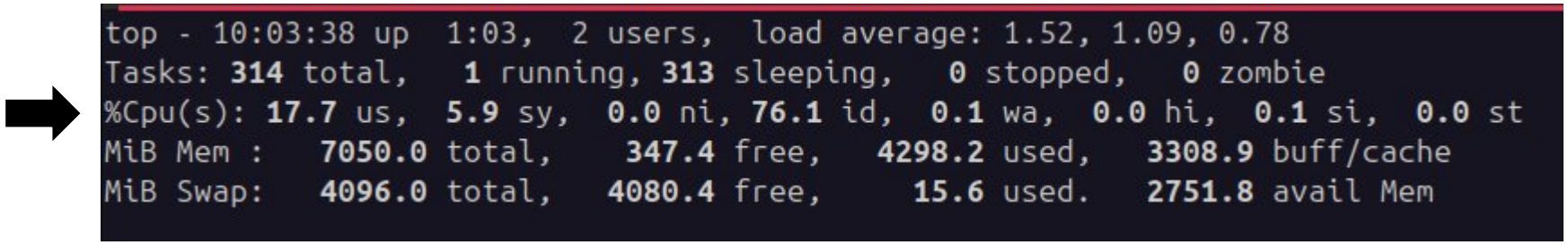


Fig 9. *top command headers*

- us (User) : Temps CPU passé dans l'espace utilisateur (vos applications).
- sy (System) : Temps CPU passé dans l'espace noyau (appels système).
- ni (Nice) : Temps CPU passé sur des processus dont la priorité a été modifiée (via nice).
- id (Idle) : Temps CPU inactif (disponible). Ici, 76.1% d'inactivité.
- wa (Wait) : Temps d'attente des entrées/sorties (disque, réseau).
- hi (Hardware IRQ) : Temps passé à gérer les interruptions matérielles.
- si (Software IRQ) : Temps passé à gérer les interruptions logicielles.
- st (Steal) : Temps "volé" par l'hyperviseur (concerne surtout les machines virtuelles).

top: Quatrième ligne : Mémoire vive (RAM)

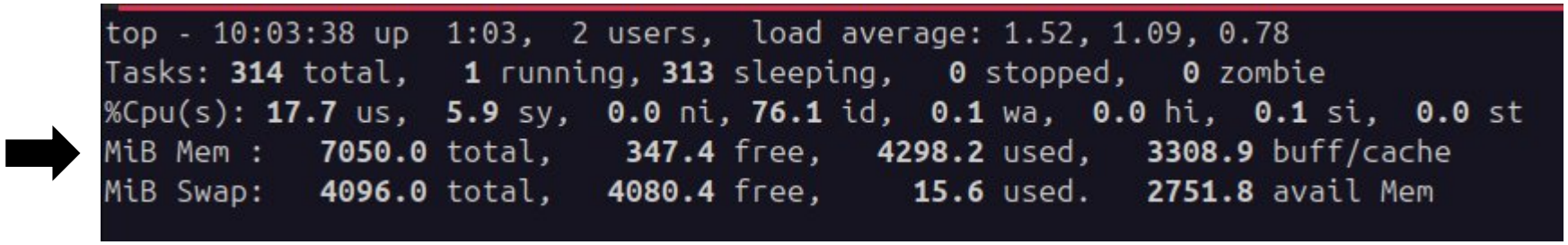


Fig 9. *top command headers*

- MiB Mem : Mémoire physique en Mebibytes (1 MiB = 1024 Ko).
- total : Quantité totale de RAM.
- free : RAM totalement inutilisée.
- used : RAM actuellement utilisée par les processus et le noyau.
- buff/cache : RAM utilisée par le noyau pour les tampons (buffers) et le cache disque.

top: Cinquième ligne : Mémoire d'échange (Swap)

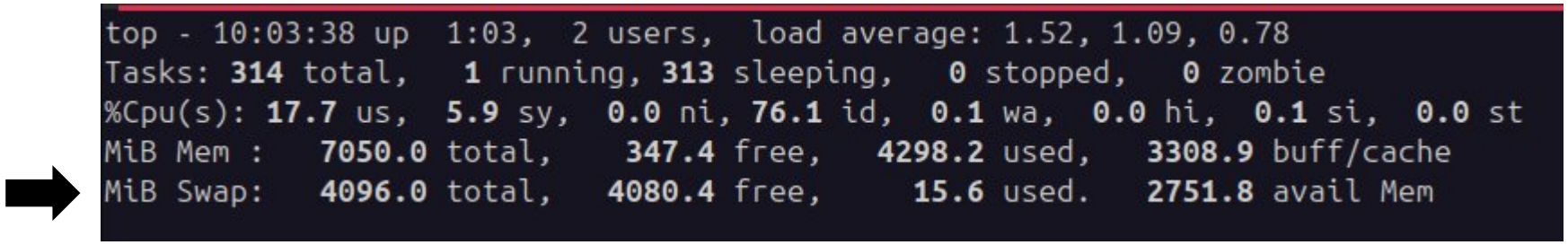


Fig 9. *top command headers*

- MiB Swap : Mémoire d'échange (sur disque) en Mebibytes.
- total : Taille totale du Swap.
- free : Swap inutilisé.
- used : Swap utilisé. Ici, seulement 15.6 MiB, c'est très bien.
- avail Mem : Mémoire disponible estimée pour lancer de nouvelles applications sans avoir à utiliser le Swap.

pstree, pgrep, pidof

- `pstree -p` : l'arbre des processus avec les PID (la figure 8)
- `pgrep motif` : trouve les PID par nom ou expression
- `pgrep -l` affiche aussi le nom
- `pgrep -u user` : filtre par utilisateur
- `pidof nom` : les PID d'un programme donné
- `$$` : PID du shell courant (echo $$)
- `$PPID` : PID de son parent
- Lancez un bash imbriqué puis `echo $$ $PPID` pour observer la relation parent-enfant

Les signaux

- **Signal** : interruption logicielle envoyée à un processus pour lui notifier un événement. Il peut être:
  - **synchrone** (ex. SIGSEGV sur accès mémoire invalide)
  - **asynchrone** (ex. SIGINT via **Ctrl+C**)
- Il peut être envoyé par le **noyau**, un **utilisateur** (commande **kill**), ou un **autre processus**
- Chaque signal a un **numéro** et un **nom** ; kill -l les liste tous
- **Action par défaut** : terminer, ignorer, ou suspendre le processus

Tableau des signaux courants

Signal	Num	Role
SIGHUP	1	Hangup / Reload. Demande à un démon de recharger sa configuration, ou indique que le terminal a été fermé.
SIGINT	2	Interrupt. Envoyé par le terminal pour arrêter le processus au premier plan (Ctrl+c)
SIGKILL	9	Force Kill. Le « sure kill ». Le noyau arrête immédiatement le processus. Ne peut pas être ignoré.
SIGTERM	15	Graceful Termination. La façon polie de demander à un processus de s'arrêter et de nettoyer.
SIGSTOP	19	Pause / Suspend. Gèle l'exécution du processus. Ne peut pas être ignoré.
SIGCONT	18	Resume. Relance un processus précédemment arrêté par SIGSTOP/SIGTSTP.

***Tab 1**. Table of common Unix/Linux signals*

kill : envoyer un signal

- kill -s signal pid ou kill -signal pid : envoyer un signal à un processus
- Sans option, kill envoie SIGTERM (15) par défaut : kill 1942 ≡ kill -15 1942
- killall nom : cible par nom (avec -i pour confirmation)
- pkill motif : par expression
- Seul **root** peut tuer les processus d'un autre **utilisateur**

Premier plan / arrière-plan

- Une commande en **premier plan** monopolise le terminal
- `commande &` : lancer directement en arrière-plan
- **Ctrl+Z** : suspendre (envoie `SIGSTOP`)
- `jobs` : lister les tâches de la session
- `bg` : reprendre en arrière-plan
- `fg` : reprendre au premier plan
- `nohup` : survivre à la déconnexion (ignore `SIGHUP`)

Section 5 : Algorithmes d'ordonnancement

Cycle CPU–E/S → préemption → dispatcher → critères → FCFS → SJF → RR → Priorité

Le cycle CPU–E/S

- L'exécution alterne des **rafales CPU** (calcul) et des **rafales E/S** (attente de périphérique)
- Le cycle commence par une rafale CPU et se termine par une requête de terminaison
- **CPU-bound** : quelques longues rafales CPU
- **I/O-bound** : beaucoup de courtes rafales
- La distribution des rafales est **exponentielle** : beaucoup de courtes, peu de longues
- Cette asymétrie est exploitée par les algorithmes que nous allons voir

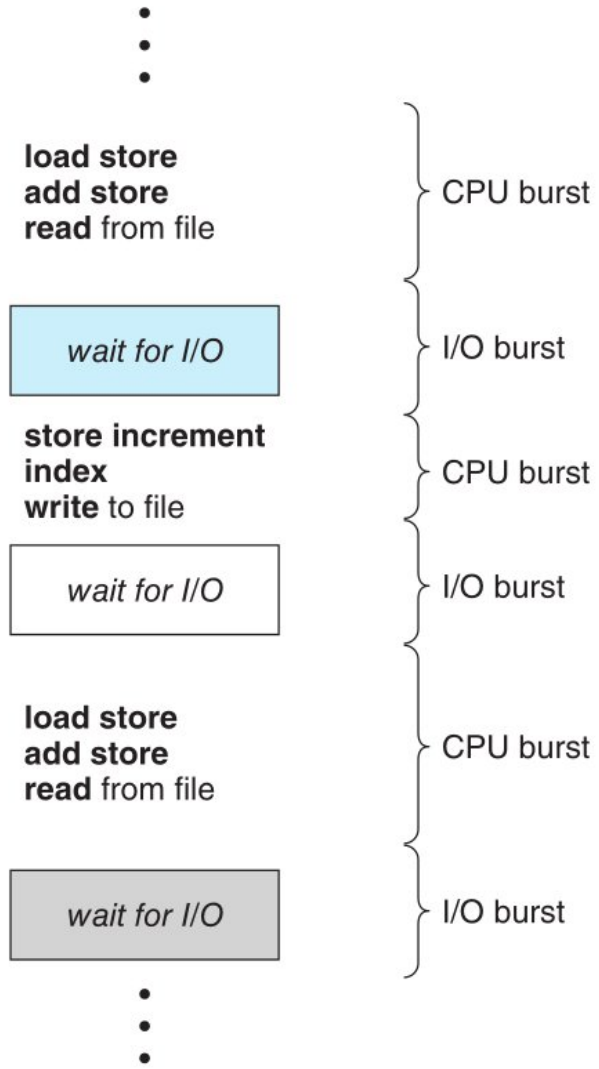


Fig 10. *Alternating sequence of CPU and I/O bursts*

Profil des Processus : I/O-bound vs CPU-bound

- **CPU-bound** : consacre plus de temps au calcul qu'aux E/S, génère peu de requêtes E/S
- **I/O-bound** : consacre plus de temps aux E/S qu'au calcul, génère beaucoup de requêtes
- **Exemple CPU-bound** : encodage vidéo, calcul scientifique
- **Exemple I/O-bound** : éditeur de texte, navigateur, traitement de fichiers
- Un bon ordonnanceur doit servir **équitablement** les deux profils

Ordonnancement non préemptif vs préemptif

Les 4 circonstances de décision d'ordonnancement du CPU:

1. Un processus passe de l'état **exécution** à l'état **attente** (ex. : requête d'E/S, appel à wait())
2. Un processus passe de l'état **exécution** à l'état **prêt** (ex. : interruption)
3. Un processus passe de l'état **attente** à l'état **prêt** (ex. : fin d'une E/S)
4. Un processus se **termine**

- Non préemptif: Cas **1** et **4** : le processus quitte le CPU de lui-même → aucun choix pour l'OS
- Préemptif: Cas **2** et **3** : le processus aurait pu continuer → l'OS choisit de lui retirer le CPU

Windows, macOS, Linux et UNIX utilisent l'ordonnancement **préemptif**

Le dispatcher

- **Dispatcher** : module qui donne le CPU au processus choisi par l'ordonnanceur
- Il effectue : la **bascule de contexte**, le **passage en mode utilisateur**, le **saut** à la bonne adresse
- **Latence de dispatch** : temps pour arrêter un processus et en démarrer un autre
- Il doit être le plus rapide possible, car invoqué à chaque bascule
- À ne pas confondre : **l'ordonnanceur choisit**, le **dispatcher applique**

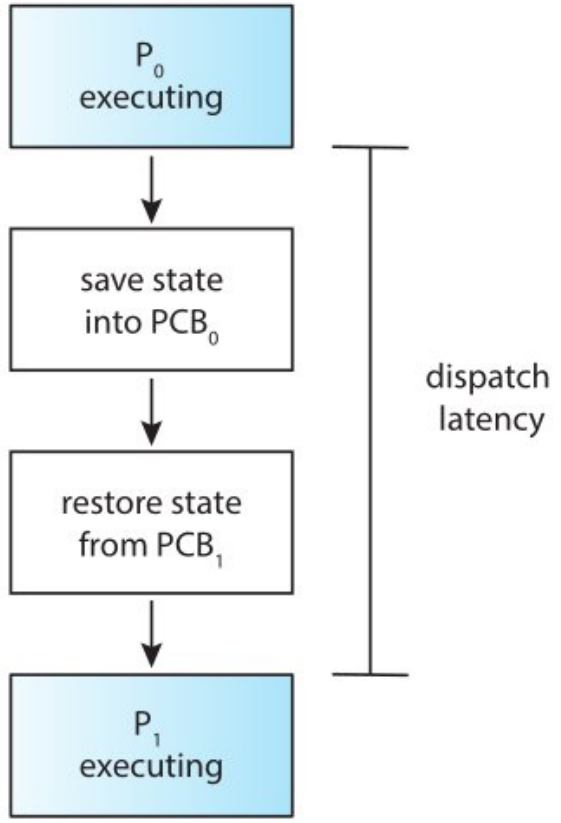


Fig 11. *The role of the dispatcher*

Critères d'ordonnancement

Pourquoi des critères ?

- Chaque algorithme a des propriétés différentes
- Un algorithme peut **favoriser** une classe de processus plus qu'une autre
- Le choix du « meilleur » algorithme dépend des critères retenus

1. **Utilisation du CPU (CPU utilization)**
   - **Objectif** : garder le CPU le plus occupé possible. Théoriquement de 0 à 100 %
   - Dans un système réel : 40 % (charge légère) à 90 % (charge lourde)
   - Mesurable avec la commande top (Linux, macOS, UNIX)
2. **Débit (throughput)**
   - Nombre de processus **terminés** par unité de temps
   - **Processus longs** : environ 1 processus toutes les quelques secondes
   - **Transactions courtes** : des dizaines de processus par seconde

Critères d'ordonnancement

3. **Temps de rotation (turnaround time)**
   - Durée entre la soumission et la terminaison d'un processus
   - = attente dans la file des prêts + exécution sur le CPU + E/S
4. **Temps d'attente (waiting time)**
   - Somme des périodes passées dans la file des prêts
   - L'algorithme n'influence ni le temps d'exécution ni le temps d'E/S
   - Il n'influence que le temps d'attente
5. **Temps de réponse (response time)**
   - Durée entre la soumission d'une requête et la première réponse
   - Important dans les systèmes interactifs
   - C'est le temps pour commencer à répondre, pas pour finir d'afficher la réponse

Recap: Critères d'ordonnancement

Critère	Mesure	Objectif
CPU utilization	% d'occupation	Maximiser
Throughput	Processus terminés / temps	Maximiser
Turnaround time	Soumission → terminaison	Minimiser
Waiting time	Temps dans la file des prêts	Minimiser
Response time	Soumission → première réponse	Minimiser

***Tab 2**. Scheduling Criteria*

FCFS : First Come, First Served

Principe:

- Algorithme d'ordonnancement le plus simple
- Le processus qui demande le CPU en premier l'obtient en premier
- Implémenté avec une file FIFO

Fonctionnement de la file:

- Un processus entre dans la file des prêts → son PCB est ajouté à la queue
- Le CPU est libre → il est alloué au processus en tête de file
- Le processus en cours d'exécution est retiré de la file

Avantage: Code simple à écrire et à comprendre

Inconvénient: Le temps d'attente moyen est souvent très long

FCFS : Exemple

Processus arrivant à t = 0:

Processus	Temps de la rafale CPU (ms)
P1	24
P2	3
P3	3

Cas 1 : ordre P1, P2, P3:

- Diagramme de Gant:



- Le temps d'attente moyen = (0 + 24 + 27) / 3 = 17 ms

Cas 2 : ordre P2, P3, P1:

- Diagramme de Gant:



- Le temps d'attente moyen = (6 + 0 + 3) / 3 = 3 ms

Le temps d'attente moyen de FCFS varie beaucoup si **les durées des rafales CPU sont très différentes**

FCFS : L'effet de convoi (convoy effect)

- CFS est **non préemptif**: Le processus garde le CPU jusqu'à sa terminaison ou une demande d'E/S
- Un processus **CPU-bound** monopolise le CPU
- Tous les processus **I/O-bound** terminent leur E/S et s'entassent en file des prêts
- Les périphériques restent **inactifs** pendant que le « gros » processus calcule
- Quand il part enfin en E/S, les petits s'exécutent vite, puis le CPU se vide
- Résultat : utilisation du CPU **et** des périphériques dégradée

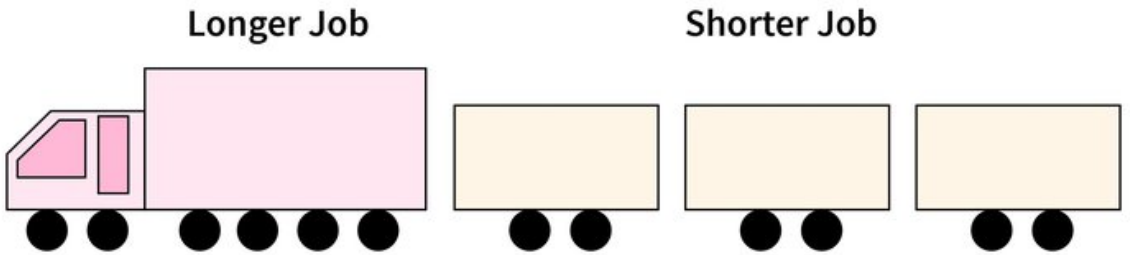


Fig 12. *Convoy effect in FCFS*

SJF: Shortest-Job-First

Principe:

- Chaque processus est associé à la longueur de sa **prochaine** rafale CPU
- Quand le CPU est libre, il est attribué au processus dont la prochaine rafale est la plus petite
- En cas d'égalité entre deux processus : on départage avec FCFS

Précision importante sur le nom:

- Le terme plus juste serait « shortest-next-CPU-burst » (plus courte prochaine rafale CPU)
- **L'ordonnancement dépend de la prochaine rafale, pas de la durée totale du processus**
- On garde le nom SJF car c'est le terme utilisé par la plupart des auteurs et manuels

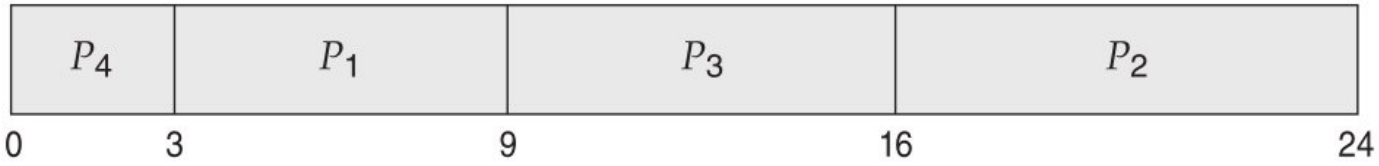
SJF: Exemple

Processus arrivant à t = 0:

Processus	Temps de la rafale CPU (ms)
P1	6
P2	8
P3	7
P4	3

Ordre SJF - P4, P1, P2, P3:

- Diagramme de Gant:



- Le temps d'attente moyen = (3 + 16+ 9 + 0) / 4 = 7 ms
- Si on utilise FCFS: Le temps d'attente moyen = 10.25 ms

SJF réduit le temps d'attente moyen d'environ 32 %

SJF: optimal… mais impossible à implémenter

SJF est optimal:

- Il donne le temps d'attente moyen minimal pour un ensemble de processus donné (prouvé)
- Idée : placer un processus court avant un long réduit l'attente du court plus qu'elle n'augmente celle du long

Probleme: Il est impossible de connaître la longueur de la prochaine rafale CPU

Solution : approximer SJF

- On ne connaît pas la **prochaine rafale**, mais on peut la **prédire**
- On choisit le processus avec la plus courte rafale **prédite**

SJF: Prédiction par moyenne exponentielle

Formule:

$$\tau_{n+1} = \alpha\, t_n + (1 - \alpha)\tau_n.$$

Symbole	Signification
t_n	Longueur mesurée de la n-ième rafale CPU (information la plus récente)
τ_n	Valeur prédite pour cette rafale (historique passé)
τ_{n+1}	Prédiction pour la prochaine rafale
α	Poids relatif du récent et du passé, avec 0 ≤ α ≤ 1

Symbole	Résultat	Hypothèse
α = 0	$\tau_{n+1} = \tau_n$	L'historique récent n'a aucun effet (conditions transitoires)
α = 1	$\tau_{n+1} = t_n$	Seule la dernière rafale compte (historique ancien et sans intérêt)
α = 1/2	Poids égaux	Cas le plus courant

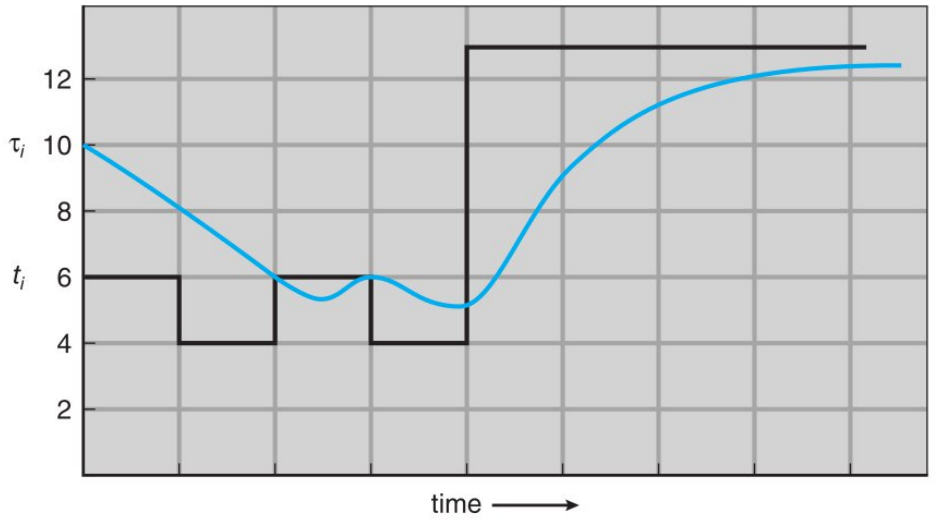


Fig 13. *Prediction of the length of the next CPU burst*
Case where: α=*1/2*, τ_0=10

Round Robin (RR) / tourniquet

Principe:

- Similaire à **FCFS**, mais avec **préemption** pour alterner entre les processus
- Un **quantum de temps** (**time slice**) est défini : généralement **10 à 100 ms**
- La file des prêts est traitée comme une **file circulaire**
- Le CPU est alloué à chaque processus pendant **1 quantum au max**

Fonctionnement:

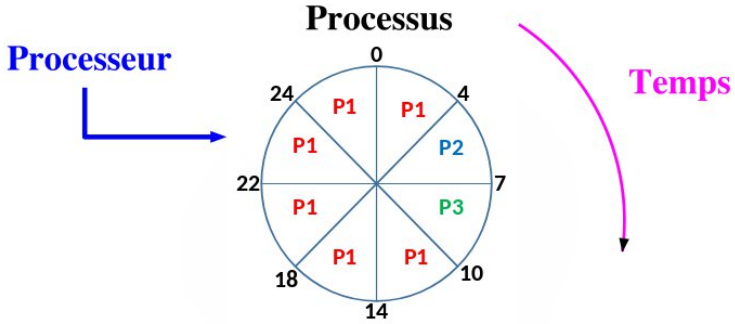
- Les nouveaux processus sont ajoutés à la **queue** de la file **FIFO**
- L'ordonnanceur prend le **premier** processus de la file
- Il programme une **minuterie** (timer) pour interrompre après 1 quantum
- **Il lance** le processus (dispatch)

Exemple: Round Robin (RR) / tourniquet

Processus arrivant à t = 0:

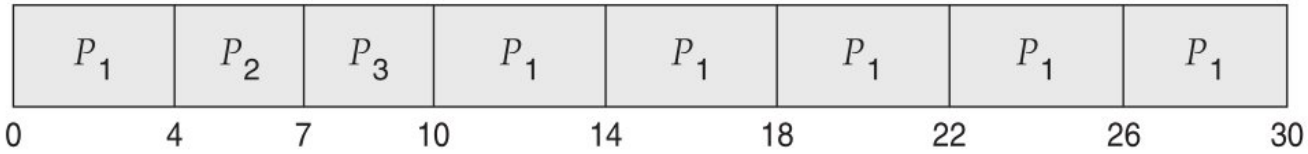
- **quantum = 4 ms**

Processus	Temps de la rafale CPU (ms)
P1	24
P2	3
P3	3



Ordre RR - P1, P2, P3, P1, P1, P1, P1, P1:

- Diagramme de Gant:



- Le temps d'attente moyen = **RR**: ((10-4) + 4 + 7) / 3 = 5.66 ms, **FCFS**: (0 + 24 + 27) / 3 = 17 ms

- **Rafale < 1 quantum**: Le processus libère le CPU volontairement → l'ordonnanceur passe au suivant
- **Rafale > 1 quantum**: La minuterie déclenche une interruption → changement de contexte → le processus est remis à la queue de la file → le suivant est sélectionné

Garanties du RR et choix du quantum

Le choix du quantum est déterminant:

- Aucun processus n'a le CPU plus d'1 quantum d'affilée (sauf s'il est seul)
- Avec n processus et un quantum q : chacun reçoit 1/n du CPU, par blocs d'au plus q
- Attente maximale avant le prochain quantum : (n – 1) × q

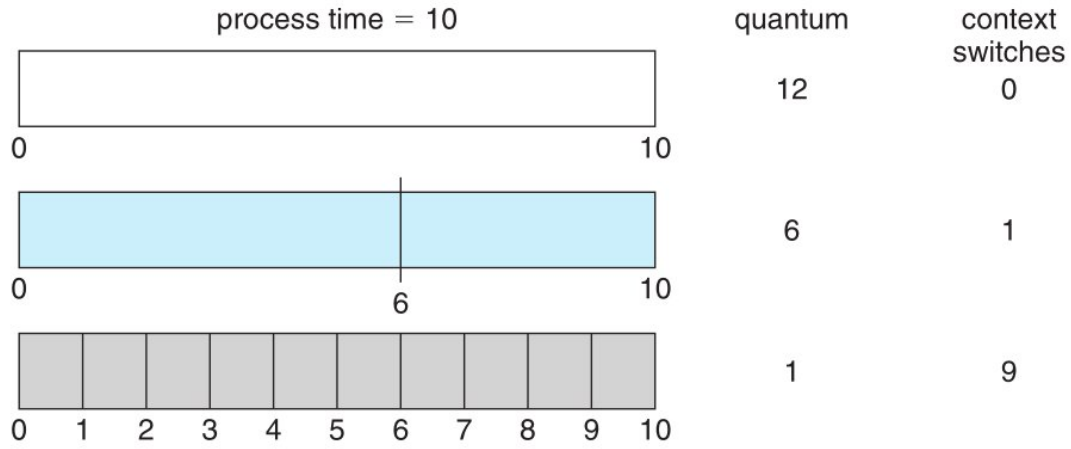


Fig 15. *How a smaller time quantum increases context switches*

Effet de la taille du quantum:

- **Très grand RR**: devient identique à FCFS
- **Très petit (ex. 1 ms)**: Beaucoup de changements de contexte → surcoût

Règles pratiques

- Le quantum doit être grand par rapport au temps de changement de contexte
- Si le changement de contexte = 10 % du quantum → ~10 % du CPU perdu
- En pratique : quantum de 10 à 100 ms, changement de contexte < 10 µs

Priority Scheduling

Principe:

- Une **priorité** est associée à chaque processus
- Le CPU est alloué au processus de **plus haute** priorité
- À priorité égale : ordre **FCFS**

Attention à la convention des nombres:

- Priorités indiquées par une plage fixe : 0 à 7, 0 à 4095, etc.
- Aucun accord général : certains systèmes utilisent les **petits** nombres pour la **basse** priorité, d'autres pour la **haute**

Dans ce cours : **petit nombre** = **haute priorité**

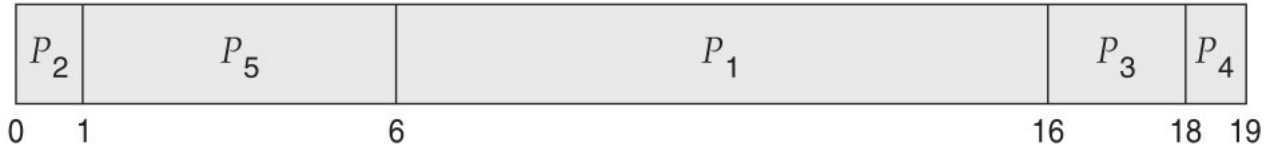
Exemple: Priority Scheduling

Processus arrivant à t = 0:

Processus	Temps de la rafale CPU (ms)	Priorité
P1	10	3
P2	1	1
P3	2	4
P4	1	5
P5	5	2

Ordre Priority - P2, P5, P1, P3, P4:

- Diagramme de Gant:



- Le temps d'attente moyen = **Priority** : (0 + 1 + 6 + 16 + 18) / 5 = 8.2 ms
- On ordonne par priorité, pas par longueur de rafale. P4 est très court (1 ms) mais passe en dernier à cause de sa priorité 5.

Priorités et préemption

Priorités définies en interne (quantités mesurables par le système)

- Limites de temps
- Besoins en mémoire
- Nombre de fichiers ouverts
- Rapport entre rafale d'E/S moyenne et rafale CPU moyenne

Priorités définies en externe (critères extérieurs à l'OS)

- Importance du processus
- Type et montant des fonds payés pour l'utilisation de l'ordinateur
- Département qui parraine le travail
- Autres facteurs,

Deux versions de l'algorithme:

Quand un processus arrive dans la file des prêts, sa priorité est comparée à celle du processus en cours,

Si le nouveau a une priorité plus haute:

- **Préemptif**: Il prend le CPU au processus en cours
- **Non préemptif**: Il est placé en tête de la file des prêts

Famine et vieillissement

Problème : blocage indéfini (famine, starvation):

- Un processus prêt mais en attente du CPU est considéré comme bloqué
- Des processus de basse priorité peuvent attendre indéfiniment
- Dans un système très chargé, un flux continu de processus plus prioritaires les empêche de s'exécuter

Anecdote : lors de l'arrêt de l'IBM 7094 du MIT en 1973, on aurait trouvé un processus de basse priorité soumis en 1967 qui n'avait jamais été exécuté.

Solution : le vieillissement (aging):

- Augmenter progressivement la priorité des processus qui attendent longtemps
- **Exemple** : priorités de 127 (basse) à 0 (haute)
- Toutes les secondes, le numéro de priorité d'un processus en attente diminue de 1 (donc sa priorité augmente)
- Un processus de priorité 127 atteint la priorité 0 en un peu plus de 2 minutes

Priorité combinée avec Round-Robin

Principe:

- Le système exécute toujours le processus de plus haute priorité
- Les processus de **même priorité** sont ordonnancés en Round-Robin

Exemple:

Processus arrivant à t = 0:

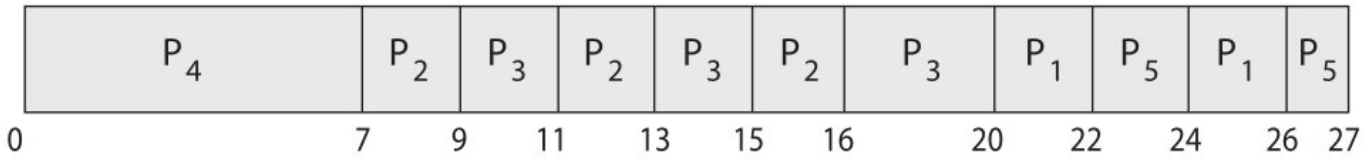
Quantum = 2 ms

Processus	Temps de la rafale CPU (ms)	Priorité
P1	4	3
P2	5	2
P3	8	2
P4	7	1
P5	3	3

- **P4** (priorité 1) : le plus prioritaire, s'exécute **jusqu'au bout** (0 à 7)
- **P2 et P3** (priorité 2) : s'alternent en Round-Robin
- À t = 16, P2 termine → P3 est **seul** à cette priorité → il s'exécute jusqu'à sa fin (20)
- **P1 et P5** (priorité 3) : s'alternent en Round-Robin jusqu'à la fin

Ordre Priority with RR:

- Diagramme de Gant:



La commande nice

nice : lancer une tâche lourde sans ralentir le reste du système

Le principe : la valeur « nice » (**gentillesse**)

- Chaque processus Linux a une valeur nice comprise entre -20 et +19
- -20 = priorité la plus haute ; +19 = priorité la plus basse, Valeur par défaut : 0
- Plus un processus est « **gentil** » (**nice élevé**), plus il laisse le CPU aux autres
- Seul **root** peut **abaisser** la valeur nice (augmenter la priorité)

Syntaxe: nice -n <valeur> <commande>

Exemples:

Commande	**Effet**
nice	Affiche la valeur nice actuelle du shell (0 par défaut)
nice -n 10 tar -czf backup.tar.gz /home	Sauvegarde avec une priorité **basse** (nice = 10)
nice -n 19 ./calcul_long.sh	Priorité **minimale** : ne gêne personne
sudo **nice** -n -10 ./traitement_urgent	Priorité **haute** (réservé à **root**)

La commande renice

renice : changer la priorité d'un processus déjà lancé

Syntaxe: renice -n <valeur> -p <PID>

Scénarios réel : une sauvegarde ralentit l'ordinateur

Étape	Commande	Effet
1. Repérer le processus gourmand	**top** (colonne NI) ou **ps -o pid,ni,cmd -p 4321**	PID = 4321, NI = 0
2. Baisser sa priorité	**renice -n 15 -p 4321**	NI passe de 0 à 15
3. Vérifier	**ps -o pid,ni,cmd -p 4321**	NI = 15 ✔

Autres usages:

Commande	Effet
renice -n 10 -u ilias	Change la priorité de tous les processus de l'utilisateur ilias
sudo renice -n -5 -p 1234	Augmente la priorité du processus 1234 (root)

Quiz: Gestion de Processus



https://forms.cloud.microsoft/r/p7qxd7jkhP